LaTeX

· ΗΥ-110 Απειροστικός Λογισμός Ι: Άσκηση Μπόνους ·

1ο Ερώτημα: Περιγραφή μεθόδου!

Η γενική εξίσωση της μεθόδου Νεωτον είναι:

$$x^{(t)} = x^{(t-1)} - a\frac{f(x^{(t-1)})}{f'^{(x^{(t-1)})}} \quad \textbf{ή} \quad x_{(t)} = x_{(t-1)} - a\frac{f(x_{(t-1)})}{f'(x_{(t-1)})}$$

Και έχουμε την συνάρτηση:

$$f(x) = x^n - A \;\; (n = 2, 3, 4)$$

Αρχικά, για την παράγωγο έχουμε:

$$f'(x) = (x^n - A)' = n * x^{n-1} - 0 = nx^{n-1}$$

Έτσι, με αντικαττάσταση θα έχουμε:

$$x^{(t)} = x^{(t-1)} - a\frac{x^{n(t-1)} - A}{nx^{n-1(t-1)}} \quad \textbf{ή} \quad x_{(t)} = x_{(t-1)} - a\frac{x^n_{(t-1)} - A}{nx^{n-1}_{(t-1)}}$$

Οπότε, για ν = 2, 3 και 4 θα είναι:

$$x_{(t)} = x_{(t-1)} - a\frac{x^2_{(t-1)} - A}{2x_{(t-1)}} \qquad x_{(t)} = x_{(t-1)} - a\frac{x^3_{(t-1)} - A}{3x^2_{(t-1)}} \qquad x_{(t)} = x_{(t-1)} - a\frac{x^4_{(t-1)} - A}{4x^3_{(t-1)}}$$

2° Ερώτημα: Υλοποίηση!

Σύμφωνα με την εκφώνηση της άσκησης, ο παρακάτω αλγόριθμος γραμμένος σε γλώσσα προγραμματισμού Python, υπολογίζει την προσεγγιστική n-οστή ρίζα του αριθμού A χρησιμοποιώντας την μέθοδο του Newton, με αρχική εκτίμηση x_0, όριο σύγκλισης tol, αριθμό επαναλήψεων $maxiter$ καθώς και συντελεστή βήματος a. Τέλος, εμφανίζει και δύο γραφήματα: εκτίμήσης και σφάλματος!

```
# ------------------------------------------------------------------------------------------ #

import matplotlib.pyplot as plt

def newtonmethod(A, n, x0, tol, maxiter, a):
    xt = x0
    estimates = [xt]
    for t in range(maxiter):
        fx = (xt ** n - A)
        primefx = (n * xt ** (n - 1))
        newxt = (xt - a * (fx / primefx))
        estimates.append(newxt)
        if (abs(newxt - xt) < tol): break
        else: xt = newxt
    print(f"\n• True {n}-th root of {A}: {A ** (1/n)} \n• Final estimate using Newton method: {xt}")
    return estimates

# [!] Example Declarations (changeable):
A, n, x0 = 1, 1, 1
# [!] Required Declarations (changeable):
tol, maxiter, a = 1e-6, 100, 0.1
estimates = newtonmethod(A, n, x0, tol, maxiter, a)

plt.get_current_fig_manager().set_window_title("• csd5748")
plt.plot(estimates, marker='o', linestyle='-', label="Approximations")
plt.axhline(y=(A ** (1/n)), color='red', linestyle='dashed', label="True n-th root")
plt.xlabel("Iteration")
plt.ylabel("Approximated Value")
plt.title(f"Convergence of Newton's Method for {n}-th Root of {A} (Step size: 0.1)")
plt.grid()
plt.legend(loc="lower right")
plt.show()

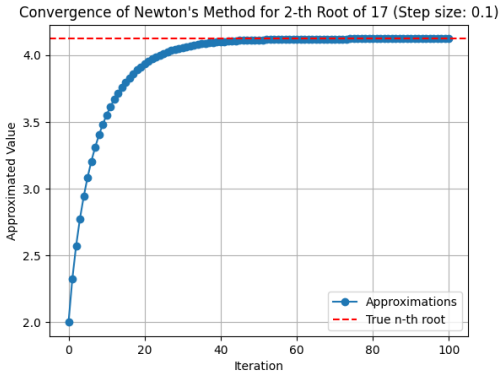
errors = [abs(estimate - (A ** (1/n))) for estimate in estimates]
plt.get_current_fig_manager().set_window_title("• csd5748")
plt.plot(errors, marker='o', linestyle='-', label="Error")
plt.yscale("log")
plt.xlabel("Iteration")
plt.ylabel("Absolute Error")
plt.title(f"Error Convergence of Newton's Method for {n}-th Root of {A} (Step size: 0.1)")
plt.grid()
plt.legend(loc="upper right")
plt.show()

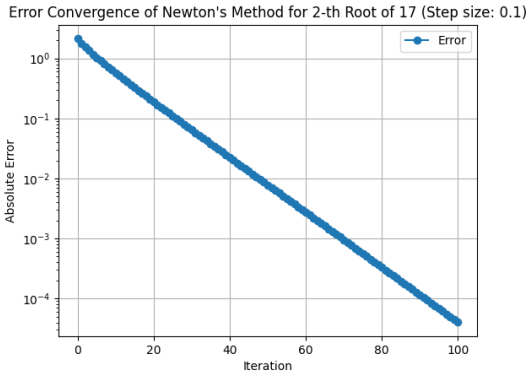
print(f"• Variables: Start Value = {x0}, Tolerance = {tol}, Loops = {len(estimates)}, Step = {a}")
print(f"• All Estimates of each Loop: {estimates}")

# ------------------------------------------------------------------------------------------ #
```

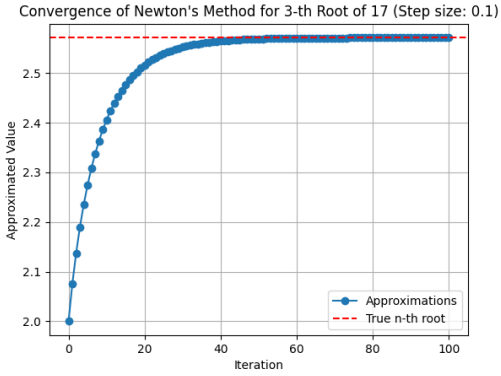
3ο Ερώτημα: Ανάλυση για Διαφορετικές Τιμές του n!

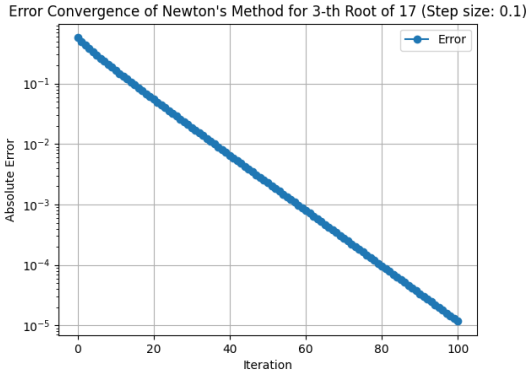
Στο συγκεκριμένο ερώτημα, θα χρησιμοποιήσω κατάλληλα τον παραπάνω αλγόριθμο. Σύμφωνα με την εκφώνηση, έχουμε τις εξής σταθερές τιμές: $A = 17$, $x_0 = 2$, $tol = 1e-6$, $a = 0.1$, και (αφου δεν αναφέρεται υποθέτω) $maxiter = 100$. Το $n = 2, 3, 4$ σημαίνει ότι έχουμε τρεις περιπτώσεις. Έτσι:



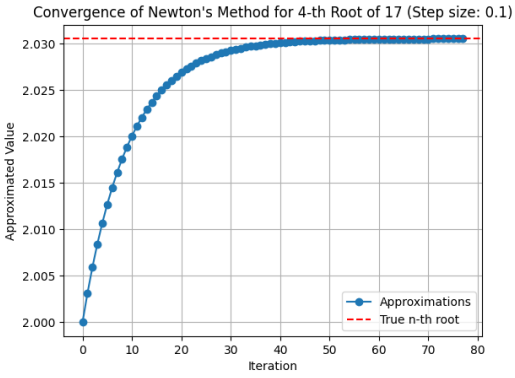


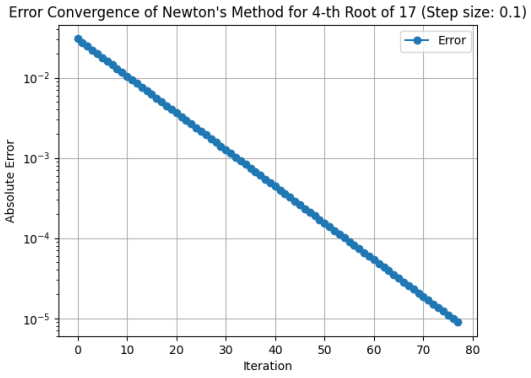
- True 2nd Root of 17: 4.123-105625617661
- Final Estimate: 4.123-065462654853





- True 3rd Root of 17: 2.5712-81590658235
- Final Estimate: 2.5712-69875277116

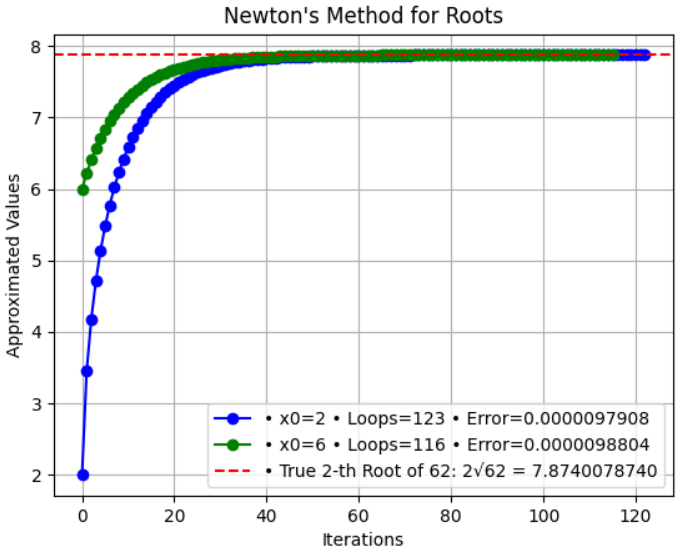


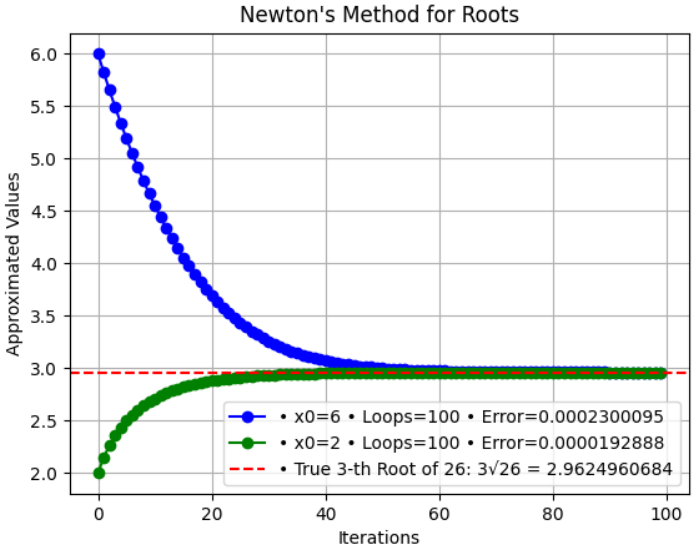


- True 4th Root of 17: 2.0305-431848689306
- Final Estimate: 2.0305-332665193307

4° Ερώτημα: Επίδραση Αρχικών Τιμών και Ανοχής!

Στο συγκεκριμένο ερώτημα, θα κάνουμε μία μικρή αλλαγή στον παραπάνω αλγόριθμο έτσι ώστε να εμφανίζονται καλύτερα τα γραφήματα. Σύμφωνα με την εκφώνηση, οι σταθερές τιμές είναι οι εξής: $tol = 1e-6$ και $a = 0.1$. Θα πρέπει να δούμε για οποιοδήποτε A αλλά και n, πώς επηρεάζεται η τελική εκτίμηση της μεθόδου $Newton$, όταν μεταβάλλονται τα x_0 και $maxiter$. Έστω τα παραδείγματα:

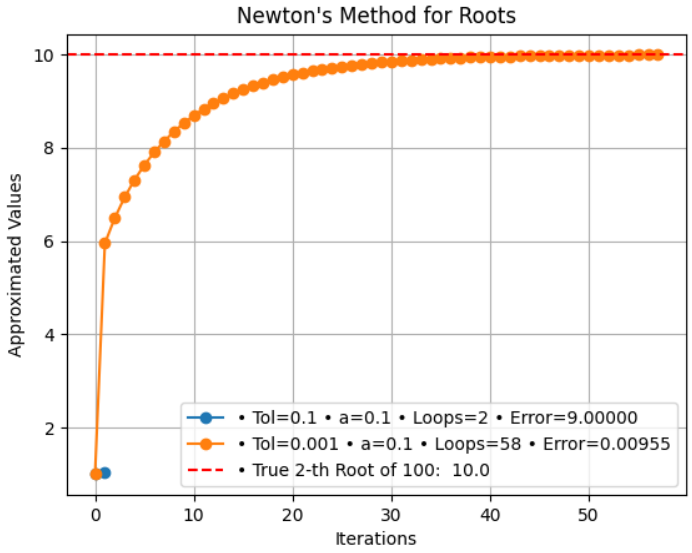


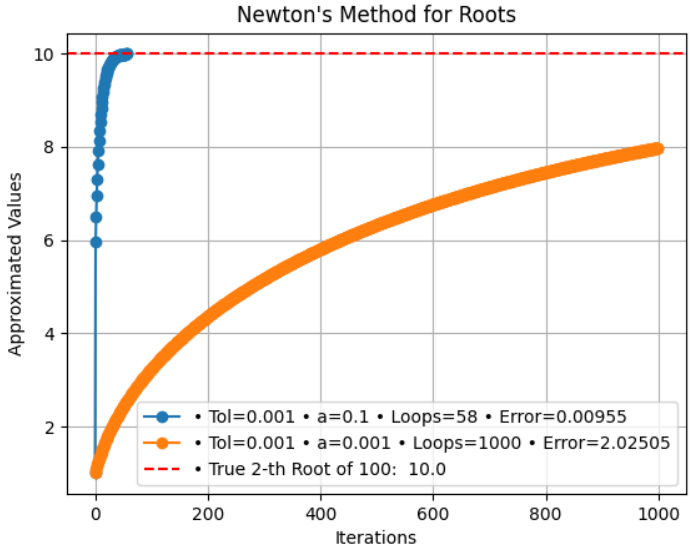


Λοιπόν, στο πρώτο σχήμα παρατηρούμε πως: όσο πιο κοντά βρίσκεται η αρχική εκτίμηση (x_0) στην πραγματική τιμή της n-οστής ρίζας του αριθμού A που μελετάμε, τόσο μικρότερος είναι ο αριθμός των επαναλήψεων ($maxiter$) που χρειάζονται για να φτάσει στην τελική εκτίμηση (με κάποιον συντελεστή βήματος a και κάποια ανοχή tol). Επιπλέον, στο δεύτερο σχήμα παρατηρούμε πως: αν ο αριθμός των επαναλήψεων ($maxiter$) είναι σταθερός, τότε το μικρότερο σφάλμα ή ανοχή ($error$) θα το έχουμε όταν η αρχική εκτίμηση (x_0) βρίσκεται πιο κοντά στην πραγματική τιμή της n-οστής ρίζας του αριθμού A που μελετάμε. Συμπέρασμα: ειναί προτειμότερο να μην έχουμε ως αρχική τιμη (x_0) κάτι τυχαίο ..!

5ο Ερώτημα: Επίδραση Αρχικών Τιμών και Ανοχής!

Στο συγκεκριμένο ερώτημα, θα κρατήσουμε την μικρή αλλαγή στον παραπάνω αλγόριθμο έτσι ώστε να εμφανίζονται καλύτερα τα γραφήματα. Σύμφωνα με την εκφώνηση, οι σταθερές τιμές (που εγώ διαλέγω) είναι οι εξής: $maxiter = 1000$, $x_0 = 1$, $A = 100$ και $n = 2$. Θα πρέπει να δούμε και να αξιολογήσουμε την ποιότητα εκτίμησης για τις διαφορετικές τιμές ανοχής tol και τις διάφορες τιμές του συντελεστη βήματος a (π.χ. **0.001**, **0.01** ή **0.1**). Έστω τα δύο παραδείγματα για **0.001** και **0.1**:





Λοιπόν, στο πρώτο σχήμα παρατηρούμε πως: όταν ο συντελεστής του βήματος a είναι σταθερός και αλλάζει μόνο η τιμή της ανοχής tol, τότε όσο μικρότερη είναι, τόσες περισσότερες επαναλήψεις $maxiter$ θα πραγματοποιηθούν έτσι ώστε να ελαχιστοποιηθεί το σφάλμα ή η απόκλιση $error$. Επιπλέον, στο δεύτερο σχήμα παρατηρούμε πως: όταν η τιμή της ανοχής tol παραμένει σταθερή, τότε όσο μικρότερος είναι ο συντελεστής του βήματος a, τόσες περισσότερες επαναλήψεις $maxiter$ (αργή σύγκλιση) θα πραγματοποιηθούν, κάνοντας τη μέθοδο του $Newton$ να συγκλίνει στην τελική της εκτίμηση. Συμπέρασμα: είναι προτιμότερες μέτριες τιμές του tol και του a, ώστε να μην έχουμε ούτε πολύ αργή ούτε και πολύ ασταθή σύγκλιση.

Bonus **Ερώτημα!**

Λοιπόν, το συγκεκριμένο «Bonus Ερώτημα» δεν ζητάει κάτι, απλά αναφέρει πώς «Υπάρχουν συγκεκριμένες τιμές για τις διάφορες παραμέτρους/τιμές, που η μέθοδος έχει «ενδιαφέρουσα» συμπεριφορά.». Όμως, επείδη ήθελα να βρω ποιες είναι αυτές οι τιμές, έφτιαξα τον παρακάτω αλγόριθμο (βασισμένος στον παραπάνω), ο οποίος για τυχαίες τιμές όλων των μεταβλητών, δημιουργεί διάφορα γραφήματα με την πραγματική τιμή της ν-οστής ρίζας ενός αρίθμου, καθώς και την εκτίμηση με βάση την μέθοδο ριζών του *Newton* (σας προτείνω να το δοκιμάσετε :) !

```
# ------------------------------------------------------------------------------------------ #

import matplotlib.pyplot as plt
import random

def newtonmethod(A, n, x0, tol, maxiter, a):
    xt = x0
    estimates = [xt]
    for t in range(maxiter):
        fx = (xt ** n - A)
        primefx = (n * xt ** (n - 1))
        newxt = (xt - a * (fx / primefx))
        estimates.append(newxt)
        if (abs(newxt - xt) < tol): break
        else: xt = newxt
    print(f"\n• The True {n}-th Root of {A}: {A ** (1/n):.0f} or {A ** (1/n)} ")
    print(f"• [NM] The Final Estimate: {xt:.0f} or {xt} ")
    print(f"• The Error = {xt - (A ** (1/n)):.0f} or {xt - (A ** (1/n))} ")
    return estimates

for i in range(random.randint(100, 1000)):
    try:
        A, n, x0 = random.randint(-1000, 1000), random.randint(0, 10), random.randint(-100, 100)
        tol, maxiter, a = random.randint(-1, 1), random.randint(10, 100), random.randint(-1, 10)
        estimates = newtonmethod(A, n, x0, tol, maxiter, a)
        print(f"• [!]: Start = {x0}, Tolerance = {tol}, Loops = {len(estimates)-1}, Steps = {a} ")
        plt.get_current_fig_manager().set_window_title("• csd5748 • Fully Randomized Newton's Roots Method
        plt.clf()
        plt.axhline(y=(A ** (1/n)), color='red', linestyle='dashed', label="!")
        plt.plot(estimates, marker='o', linestyle='-', color=(random.random(), random.random(), random.randor
        plt.title(f"• Newton's Method for Roots :D")
        plt.grid()
        plt.legend(loc="upper right")
        plt.show()
    except: pass

# A Specific Test:  A, n, x0, tol, maxiter, a = 100, 5, 100, 0, 100, ?
# ------------------------------------------------------------------------------------------ #
```

Τέλος, το δικό μου συμπέρασμα είναι πως η μέθοδος ριζών του *Newton* έχει ενδιαφέρουσα συμπεριφορά όταν ο συντελεστής βήματος a έχει συγκεκριμένες τιμές, όπως « $\mathbf{0}$ » ή « $\mathbf{2}n$ » για παράδειγμα ...

LaTeX

ΤΕΛΟΣ
(ευχαριστώ)